

Step 2 — Wrap It as a Single MCP Tool — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (real `MCPServer` code), **Advanced** (the part of this step that's easy to get subtly wrong). Read Basic first even if you already built Project 7's server — it's the fastest way to see exactly what's new here.

- [Hint 1 — The thinnest possible wrapper](#)
- [Hint 2 — Testing it, and the stdout trap again](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The Thinnest Possible Wrapper

Basic Version

`mcp_server.py` needs exactly three things: a `MCPServer` instance, one function decorated with `@mcp.tool()` that calls Step 1's `run_pipeline()`, and a line at the bottom that actually starts the server. Nothing else belongs in this file yet.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This is nearly identical in shape to Project 7's `server.py` — the same `MCPServer("some-name")`, the same `@mcp.tool()` decorator, the same `mcp.run(transport="stdio")` at the bottom. The only real difference is what's inside the function: instead of touching a database directly, it calls `run_pipeline()`, a function you already wrote and already tested in Step 1.

Pieces to use: - `from mcp.server.mcpserver import MCPServer` - `mcp = MCPServer("deep-research-service")` - `@mcp.tool()` above a function with a type-hinted signature: `def deep_research(topic: str) -> str:` - `if __name__ == "__main__":`
`mcp.run(transport="stdio")`

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The temptation at this step is to make `deep_research` 's body "smarter" right away — add the trace, add error handling, maybe even add a second tool for checking on an in-progress run. Don't. This step exists specifically to prove that the MCP layer can be *this thin* — two real lines of logic (call `run_pipeline()` , return a piece of its result) wrapped in an `@mcp.tool()` decorator. Adding more now blurs the exact lesson this step is testing, and makes Step 3's actual new work (the trace) harder to see as new.

The one thing worth doing carefully here is the tool's docstring. A caller deciding whether to use `deep_research` has no way to know, from the name alone, that it might take several seconds and run multiple model calls internally — say that plainly in the docstring, the same "a tool's description is really a prompt" lesson from Doc06, now protecting a caller from a bad surprise about *cost and time*, not just about what the tool does.

Difference between Basic, Intermediate, and Advanced: Basic names the three required pieces. Intermediate gives the exact `MCPServer` shape, reusing Project 7's pattern almost unchanged. Advanced is a warning: this step's whole point is a deliberately thin wrapper, and the real skill is resisting the urge to add more to it before Step 3 and Step 4 actually ask you to.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — Testing It, and the Stdout Trap Again

Basic Version

Run `mcp dev mcp_server.py`. This opens the MCP Inspector, the same tool Project 7 used. Find `deep_research` in its tool list, call it with a real topic, and wait — this step's version has no trace yet, so you'll see nothing until it's completely done, then just the final summary.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

If the Inspector connects but `deep_research` never shows up in the tool list, check the same two things Project 7's troubleshooting table names: the function is actually decorated with `@mcp.tool()`, and `mcp.run(transport="stdio")` is the very last line that actually executes (not inside an `if` branch that never runs, not after code that raises first).

If the Inspector hangs with no response at all, and `mcp_server.py` itself looks fine, check `researcher.py`, `fact_checker.py`, and `pipeline.py` too — a stray `print()` anywhere in that whole import chain corrupts the stdio protocol channel, not just a `print()` in `mcp_server.py` itself.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

There's a second, quieter way this step can look broken without actually being broken:

`deep_research` genuinely takes several seconds to return, because it's making 2-6 real LLM calls (2-3 rounds, 2 calls per round) before it can answer at all. If you're used to Project 7's near-instant SQLite tools, this delay can feel like a hang even when nothing is wrong. Time one call by hand (a stopwatch, or a `time.time()` print in your throwaway test script, never inside `mcp_server.py` itself) before assuming something's broken — this exact "it's slow, not stuck" problem is the entire reason Step 3 exists.

Also worth checking now, so Step 3 isn't a surprise: does `config.max_revision_rounds` actually reach `run_pipeline()`? A common mistake here is hardcoding `max_rounds=3` directly in `mcp_server.py` instead of reading it from config — it works today, but it silently ignores anything you change in `.env` later.

Difference between Basic, Intermediate, and Advanced: Basic says how to open the Inspector and what to expect. Intermediate covers the two concrete failure signatures (missing tool, total hang) and their causes, reused directly from Project 7. Advanced is about telling apart a real bug from an expected side effect of this exact step — a multi-call pipeline is genuinely slow, and that "slowness with no feedback" is a real product problem, not a bug, which is exactly what Step 3 is about to fix on purpose.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)